

Exercício — Jogo Caça ao Tesouro

Sumário

1. Objetivo.....	2
2. Tabuleiro.....	2
3. Conteúdo das casas	2
4. Configuração dos níveis	3
Regra de arredondamento.....	3
5. Geração do cenário	4
6. Sorteio dos elementos	4
Bônus	4
Risco.....	4
Tesouro	5
7. Casas sem sobreposição.....	5
8. Bateria	5
9. Função DiminuirBateria	6
10. Bônus.....	6
Bônus de 05 créditos	6
Bônus de 10 créditos	6
11. Função Bônus.....	6
12. Risco.....	6
13. Função Risco.....	7
14. Ordem de execução de uma rodada.....	7
15. Tesouro.....	7
16. Condições de encerramento	8
17. Funções obrigatórias	8
GerarCenario()	8
DiminuirBateria()	8
Bonus().....	8
Risco().....	8
18. Resultado final	8
19. Requisitos obrigatórios	9
20. Conceitos trabalhados	9
21. Entrega do trabalho	10

1. Objetivo

Desenvolver um jogo de **Caça ao Tesouro** utilizando uma **matriz bidimensional 5 × 5**, totalizando 25 casas.

O jogador inicia o percurso na primeira posição da matriz, [0,0], e percorre o tabuleiro sequencialmente até:

- encontrar o tesouro; ou
- não possuir bateria suficiente para iniciar uma nova rodada.

O exercício deverá utilizar **funções, matriz, números aleatórios, estruturas de repetição, estruturas condicionais e cálculos percentuais**.

2. Tabuleiro

O tabuleiro deverá ser representado por uma matriz de: **5 linhas × 5 colunas = 25 casas**

A matriz utiliza os índices iniciando em zero.

A numeração das casas deverá seguir a ordem:

Casa	Posição
01	[0,0]
02	[0,1]
03	[0,2]
04	[0,3]
...	...
25	[4,4]

O jogador inicia obrigatoriamente na **Casa 01 — [0,0]**.

O percurso deverá ser realizado de forma sequencial, da primeira até a última casa.

3. Conteúdo das casas

Cada casa poderá possuir somente **um tipo de conteúdo**.

Utilize as seguintes representações:

Código	Significado
---	Casa vazia
B05	Bônus de 05 créditos de bateria
B10	Bônus de 10 créditos de bateria

Código	Significado
RIS	Risco — perde 03 créditos de bateria
\$\$\$	Tesouro

O cenário deverá possuir obrigatoriamente:

- 21 casas vazias;
- 01 casa com bônus de 05 créditos;
- 01 casa com bônus de 10 créditos;
- 01 casa com risco;
- 01 casa com tesouro.

Total: 25 casas.

4. Configuração dos níveis

No início da execução, o sistema deverá solicitar ao usuário os percentuais correspondentes aos três níveis do jogo:

Percentual do Nível I:

Percentual do Nível II:

Percentual do Nível III:

Os três percentuais deverão totalizar **exatamente 100%**.

Caso a soma seja diferente de 100%, o sistema deverá informar que os valores são inválidos e solicitar novamente os percentuais.

A quantidade de casas pertencente a cada nível deverá ser calculada pelo sistema com base nos percentuais informados pelo usuário.

O tabuleiro possui 25 casas.

Nível	Percentual	Casa
Nível I	25%	casas 01 a 06 = 06 casas
Nível II	35%	casas 07 a 15 = 09 casas
Nível III	40%	casas 16 a 25 = 10 casas

O sistema deverá calcular os limites inferior e superior (primeira e última casa do nível):

Nível I: $25 \times 25\% = 6,25 \rightarrow$ (casas 01 a 06 = **06 casas**)

Nível II: $25 \times (25\% + 35\%) = 15,00 \rightarrow$ (casas 07 a 15 = **09 casas**)

Nível III: $25 \times 100\% = 25 \rightarrow$ (casas 16 a 25 = **10 casas**)

Regra de arredondamento

Quando o cálculo resultar em número decimal, deverá ser aplicado o arredondamento convencional:

6,2 → 6
7,8 → 8

O sistema deverá calcular automaticamente o início e o fim de cada nível.

Os níveis não devem ser definidos manualmente no código.

5. Geração do cenário

A função:

`GerarCenario()`

será responsável pela criação do cenário do jogo.

Essa função deverá:

1. criar/inicializar a matriz;
2. calcular os limites dos três níveis;
3. atribuir o nível correspondente a cada casa;
4. sortear a posição do bônus de 05 créditos;
5. sortear a posição do bônus de 10 créditos;
6. sortear a posição do risco;
7. sortear a posição do tesouro;
8. garantir que nenhuma posição seja utilizada por mais de um elemento;
9. garantir as restrições de nível dos elementos.

A geração dos elementos deverá ocorrer **depois da definição dos níveis**.

6. Sorteio dos elementos

O cenário deverá sortear aleatoriamente:

Bônus

- 01 casa com bônus de 05 créditos;
- 01 casa com bônus de 10 créditos.

Os bônus poderão estar em qualquer nível.

Risco

Deverá existir apenas **01 casa de risco**.

O risco:

- deverá ser sorteado aleatoriamente;
- não poderá estar no Nível I;
- poderá estar no Nível II ou Nível III.

Tesouro

Deverá existir apenas **01 casa com o tesouro**.

O tesouro:

- deverá ser sorteado aleatoriamente;
- não poderá estar no Nível I;
- deverá estar no Nível II ou Nível III;
- não poderá ocupar uma casa que já contenha bônus;
- não poderá ocupar uma casa que já contenha risco.

7. Casas sem sobreposição

Uma casa poderá possuir somente um conteúdo.

Portanto, não é permitido:

B05 + RIS
B10 + RIS
B05 + \$\$\$
RIS + \$\$\$

Por exemplo, se o sorteio determinar que o bônus B05 ficará na Casa 12, nenhum outro elemento poderá ocupar a Casa 12.

Caso uma posição sorteada já esteja ocupada, o sistema deverá realizar um novo sorteio.

8. Bateria

O jogador inicia o jogo com:

Bateria = 100 créditos

Cada rodada possui um custo de:

10 créditos de bateria

Portanto, para iniciar uma nova rodada, o jogador deverá possuir pelo menos:

10 créditos

A regra geral para continuar o jogo é:

Bateria $\geq$ 10

Caso a bateria seja inferior a 10 créditos, uma nova rodada não poderá ser iniciada e o jogo será encerrado.

A bateria poderá ultrapassar 100 créditos quando o jogador receber bônus.

9. Função DiminuirBateria

Deverá ser criada uma função:

`DiminuirBateria()`

Essa função deverá realizar o consumo de bateria correspondente à rodada.

A cada rodada:

`Bateria = Bateria - 10`

10. Bônus

Quando o jogador entrar em uma casa que contenha um bônus, o valor deverá ser automaticamente convertido em bateria.

Bônus de 05 créditos

`Bateria = Bateria + 5`

Bônus de 10 créditos

`Bateria = Bateria + 10`

Também deverá ser contabilizado o total de créditos obtidos pelo jogador.

Não será necessário solicitar nenhuma decisão ao jogador ao encontrar um bônus.

O bônus é aplicado automaticamente.

11. Função Bônus

Deverá ser criada uma função:

`Bonus()`

A função deverá receber o valor do bônus encontrado e realizar a conversão do bônus em créditos de bateria.

Exemplo conceitual:

`Bonus(5)`

`Bonus(10)`

A implementação deverá utilizar parâmetros para receber o valor do bônus.

12. Risco

O cenário deverá possuir uma única casa contendo o risco.

Ao entrar nessa casa, o jogador perderá:

3 créditos de bateria

O risco será aplicado **depois do consumo normal da rodada**.

Portanto, uma rodada em uma casa de risco terá:

Consumo da rodada: -10
Risco: -3
Total: -13

13. Função Risco

Deverá ser criada uma função: `Risco()`

Essa função deverá aplicar a penalidade correspondente ao risco:

`Bateria = Bateria - 3`

14. Ordem de execução de uma rodada

A cada rodada, o sistema deverá seguir a seguinte lógica:

1. verificar se existe bateria suficiente para iniciar a rodada;
2. avançar para a casa correspondente;
3. diminuir 10 créditos de bateria;
4. verificar o conteúdo da casa;
5. se for um bônus, aplicar o bônus;
6. se for risco, aplicar a penalidade de 3 créditos;
7. se for o tesouro, encerrar imediatamente o jogo;
8. registrar a rodada;
9. continuar para a próxima casa.

A verificação do conteúdo da casa deverá ocorrer **após o consumo dos 10 créditos da rodada**.

15. Tesouro

O tesouro é o objetivo principal do jogo.

Ao localizar uma casa contendo: \$\$\$

o jogo deverá ser encerrado imediatamente.

O jogador será considerado vencedor.

O resultado deverá registrar: **Tesouro encontrado: SIM**

16. Condições de encerramento

O jogo poderá terminar de duas formas.

Critério	Descrição	Resultado
Vitória	O jogador encontra o tesouro	Tesouro encontrado: SIM
Derrota	O jogador não possui pelo menos 10 créditos de bateria para iniciar uma nova rodada	Tesouro encontrado: NÃO

Como o tesouro é obrigatoriamente colocado entre as casas dos Níveis II e III, a geração correta do cenário garante que ele estará disponível antes do final do tabuleiro.

17. Funções obrigatórias

O programa deverá utilizar, no mínimo, as seguintes funções:

`GerarCenario()`

Responsável por criar e configurar o cenário do jogo.

`DiminuirBateria()`

Responsável por diminuir 10 créditos de bateria a cada rodada.

`Bonus()`

Responsável por converter o bônus encontrado em créditos de bateria.

`Risco()`

Responsável por aplicar a penalidade de 3 créditos de bateria.

As funções deverão ser utilizadas efetivamente pelo programa.

18. Resultado final

Ao finalizar o jogo, o programa deverá apresentar o cenário e as informações do resultado.

Exemplo:

```
===== RESULTADO DO JOGO =====
```

```
--- --- B05 --- ---
--- RIS --- ---
--- --- B10 ---
--- --- --- ---
--- --- $$$ ---
```

Bateria restante: 67 créditos

Créditos obtidos: 15 créditos
Nível atingido: II
Tesouro encontrado: SIM

Posição do risco:
Casa: 07

Quantidade de rodadas: 17

=====

Os valores apresentados acima são apenas **exemplos de saída**. O cenário, as posições e os resultados deverão ser gerados pelo programa.

19. Requisitos obrigatórios

O programa deverá obrigatoriamente:

- utilizar uma matriz 5×5 ;
- utilizar índices de matriz iniciando em zero;
- iniciar na posição $[0, 0]$;
- percorrer as 25 casas sequencialmente;
- solicitar os percentuais dos três níveis;
- validar que a soma dos percentuais seja exatamente 100%;
- calcular automaticamente os limites dos níveis;
- aplicar a regra de arredondamento definida;
- utilizar geração de números aleatórios;
- sortear os dois bônus;
- sortear o risco;
- sortear o tesouro;
- impedir que dois elementos ocupem a mesma casa;
- impedir risco e tesouro no Nível I;
- utilizar as funções obrigatórias;
- consumir 10 créditos de bateria por rodada;
- aplicar o risco após o consumo da rodada;
- aplicar automaticamente os bônus encontrados;
- encerrar o jogo ao encontrar o tesouro;
- encerrar o jogo quando não houver bateria suficiente para uma nova rodada;
- apresentar o resultado final.

20. Conceitos trabalhados

Este exercício deverá permitir a aplicação prática dos seguintes conceitos:

- declaração e utilização de matrizes;
- índices de linhas e colunas;
- estruturas de repetição;
- estruturas condicionais;
- funções;
- parâmetros de funções;
- retorno de funções, quando necessário;
- variáveis acumuladoras;
- números aleatórios;
- validação de dados;
- cálculos percentuais;
- arredondamento;
- controle de estado do jogo;
- manipulação de dados armazenados em matriz.

21. Entrega do trabalho

O exercício deverá ser entregue (postar o link do repositório do github) na atividade do classroom.

O repositório deve constar as evoluções do projeto, considerando as seguintes etapas:

1. Criação do repositório
2. Evolução do desenvolvimento, atendendo os requisitos obrigatórios. Como sugestão, utilizar um checklist para:

- utilizar uma matriz 5×5 ;
- solicitar os percentuais dos três níveis;
- validar que a soma dos percentuais seja exatamente 100%;
- calcular automaticamente os limites dos níveis;
- aplicar a regra de arredondamento definida;
- utilizar geração de números aleatórios;
- sortear os dois bônus;
- sortear o risco;
- sortear o tesouro;
- impedir que dois elementos ocupem a mesma casa;
- impedir risco e tesouro no Nível I;
- utilizar as funções obrigatórias;
- consumir 10 créditos de bateria por rodada;
- aplicar o risco após o consumo da rodada;
- aplicar automaticamente os bônus encontrados;
- encerrar o jogo ao encontrar o tesouro;
- encerrar o jogo quando não houver bateria suficiente para uma nova rodada;

- apresentar o resultado final.
3. Considerar as funções obrigatórias:
- GerarCenario()
 - DiminuirBateria()
 - Bonus()
 - Risco()
4. Documentação do jogo.